

Teach Module 51 README

Module 51: Capstone Security, CI/CD and Deployment

This README is a study and teaching guide for Module 51. The course document was used only to identify the module topic list. The explanations, examples, exercises, and lab steps below are original teaching material.

Learning Goal

By the end of this module, you should understand how to move a capstone project from "the application works" to "the application is secure, automatically built, deployed, and verified."

The professional delivery flow is:

```
Code -> Security checks -> Build -> Test -> Package -> Provision infrastructure -> Deploy -> Verify
```

1. Security Testing: SAST And DAST

Security testing helps catch weaknesses before a system is released.

SAST

SAST means Static Application Security Testing.

It scans source code before the application runs.

SAST asks:

```
Does the code contain dangerous patterns?
```

Example risky code:

```
String query = "SELECT * FROM users WHERE name = '" + username + "'";
```

This may allow SQL injection because user input is directly inserted into the SQL statement.

A safer approach uses parameterized queries:

```
PreparedStatement stmt =  
    connection.prepareStatement("SELECT * FROM users WHERE name = ?");  
stmt.setString(1, username);
```

Common SAST tools include:

```
SonarQube  
Semgrep  
Checkmarx  
Veracode
```

```
GitHub CodeQL
SpotBugs
```

DAST

DAST means Dynamic Application Security Testing.

It scans a running application from the outside, similar to how an attacker would test the system.

DAST asks:

```
When the app is live, can someone exploit it?
```

Example DAST findings:

```
Missing security headers
Exposed admin routes
Weak authentication behavior
Cross-site scripting vulnerabilities
Insecure cookies
```

Common DAST tools include:

```
OWASP ZAP
Burp Suite
StackHawk
```

The simple difference:

```
SAST = checks the code
DAST = checks the running app
```

2. CI/CD Pipeline Execution

CI/CD means Continuous Integration and Continuous Delivery or Deployment.

In a Java project, a pipeline usually performs these steps:

```
Checkout code
Set up Java
Install dependencies
Run unit tests
Run integration tests
Build JAR or container image
Run security scans
Push artifact or image
Deploy to environment
Run smoke tests
```

A simplified GitHub Actions workflow:

```
name: Java CI

on:
  push:
    branches: [ main ]

jobs:
  build-test-scan:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
        run: mvn test

      - name: Build app
        run: mvn package
```

The important mindset:

The pipeline should prove the application is safe enough to deploy without relying on manual memory.

3. Infrastructure Provisioning With Terraform And Ansible

Terraform and Ansible both automate infrastructure work, but they solve different problems.

Terraform

Terraform is mainly used to create infrastructure.

Examples:

```
Create a cluster
Create a network
Create a database
Create cloud resources
Create storage
```

Terraform is declarative. You describe what should exist.

Example:

```
resource "aws_s3_bucket" "app_bucket" {  
  bucket = "my-capstone-app-bucket"  
}
```

Terraform asks:

What infrastructure should exist?

Ansible

Ansible is mainly used to configure environments.

Examples:

```
Install Java  
Install Docker  
Configure services  
Copy files  
Start an application  
Apply server settings
```

Ansible asks:

How should this environment be configured?

The simple distinction:

```
Terraform = provision infrastructure  
Ansible = configure infrastructure
```

4. OpenShift Deployment

OpenShift is Red Hat's Kubernetes-based platform for running containerized applications.

A common Java deployment path looks like this:

```
Java source code -> JAR file -> Docker/OCI image -> OpenShift deployment
```

Example Dockerfile:

```
FROM eclipse-temurin:17-jre  
COPY target/app.jar app.jar  
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Important OpenShift concepts:

```
Project/Namespace = logical workspace  
Deployment = desired app runtime definition
```

```
Pod = running instance of your app
Service = stable internal network access
Route = external URL to reach the app
ConfigMap = non-secret configuration
Secret = sensitive configuration
```

Request flow:

```
User -> OpenShift Route -> Service -> Pod -> Java app
```

Deployment verification should answer:

```
Did the pod start?
Are logs clean?
Is the route reachable?
Can the app connect to its database?
Do health endpoints pass?
```

5. Smoke Testing And Readiness

Smoke testing is a small set of checks that confirms a deployment is basically alive.

Smoke tests do not replace full testing.

They ask:

```
Did we deploy something usable, or is it obviously broken?
```

Example smoke tests:

```
GET /actuator/health returns 200
Login page loads
Main API endpoint responds
Database-backed endpoint works
No startup crash in logs
```

For a Spring Boot app, a common endpoint is:

```
/actuator/health
```

Example check:

```
curl -f https://my-app.example.com/actuator/health
```

If that command fails, the deployment should be considered unhealthy.

Module 51 Big Picture

Before this module:

You built the app.

After this module:

You can prove the app is secure enough, deployable, repeatable, and working in a real environment.

Final delivery flow:

Developer pushes code
Pipeline runs tests
Pipeline runs SAST
Pipeline builds artifact/container
Pipeline deploys to OpenShift
DAST scans running app
Smoke tests verify deployment
Team confirms readiness

Practice Questions

1. What is the difference between SAST and DAST?
2. Why should security scanning happen inside the CI/CD pipeline?
3. What does Terraform do that Ansible usually does not?
4. What is the role of an OpenShift Route?
5. Why are smoke tests useful after deployment?

Short answers:

1. SAST scans source code; DAST scans the running app.
2. To catch security issues automatically before release.
3. Terraform creates infrastructure resources.
4. A Route exposes an OpenShift service externally.
5. They quickly confirm the deployed app is basically working.

Practice Exercises

Exercise 1: Add SAST To Your Project

Pick a Java/Spring Boot project and run a static security scan.

Practice tasks:

Run a tool like Semgrep, SonarQube, SpotBugs, or GitHub CodeQL
Identify at least 3 findings
Classify each finding as high, medium, or low risk
Fix at least 1 real issue
Document any false positives

Deliverable:

```
security-sast-report.md
```

Exercise 2: Run DAST Against A Running App

Start your application locally or in a test environment, then scan it with OWASP ZAP or a similar tool.

Practice tasks:

```
Start the application
Run a baseline DAST scan
Review alerts
Identify missing security headers or exposed endpoints
Fix one configuration issue
Run the scan again
```

Deliverable:

```
security-dast-report.md
```

Exercise 3: Build A CI Pipeline

Create a CI workflow that runs automatically when code is pushed.

Pipeline should include:

```
Checkout source code
Set up Java
Run unit tests
Build the application
Save the build artifact
```

Target file:

```
.github/workflows/ci.yml
```

Exercise 4: Add Security Gates To CI/CD

Extend the CI pipeline so security checks can block bad releases.

Add steps for:

```
Dependency vulnerability scan
SAST scan
Test execution
Application build
```

Optional tools:

OWASP Dependency-Check
Snyk
Trivy
Semgrep
CodeQL
SonarQube

Deliverable:

pipeline-security-policy.md

Exercise 5: Containerize The Java App

Create a container image for the capstone application.

Practice tasks:

Build the JAR
Create a Dockerfile
Build the image
Run the container locally
Confirm the app starts
Call the health endpoint

Example commands:

```
docker build -t capstone-app .  
docker run -p 8080:8080 capstone-app  
curl http://localhost:8080/actuator/health
```

Exercise 6: Scan The Container Image

Once the image is built, scan it for vulnerabilities.

Tools:

Trivy
Grype
Docker Scout
Snyk Container

Practice tasks:

Scan the image
Identify vulnerable packages
Rebuild with a safer base image if needed
Compare before/after results

Deliverable:

```
container-security-report.md
```

Exercise 7: Provision Infrastructure With Terraform

Write a small Terraform configuration for a simple environment.

Practice with:

```
Local Docker provider
Cloud VM
Storage bucket
Database instance
Network resource
```

Practice tasks:

```
Write main.tf
Run terraform init
Run terraform plan
Run terraform apply
Inspect created resources
Run terraform destroy
```

Deliverable:

```
infra/
  main.tf
  variables.tf
  outputs.tf
```

Exercise 8: Configure An Environment With Ansible

Use Ansible to configure a server-like environment.

Practice tasks:

```
Install Java
Install Docker or Podman
Create an application directory
Copy configuration files
Start a service
```

Deliverable:

```
ansible/
  inventory.ini
  playbook.yml
```

Exercise 9: Deploy To OpenShift

Deploy the capstone application to OpenShift.

Practice tasks:

```
Create or select a project
Deploy the container image
Create a service
Expose a route
Set environment variables
Check pod logs
Verify the route URL
```

Useful commands:

```
oc new-project capstone
oc apply -f deployment.yaml
oc apply -f service.yaml
oc expose service capstone-app
oc get pods
oc logs deployment/capstone-app
```

Deliverable:

```
openshift/
  deployment.yaml
  service.yaml
  route.yaml
```

Exercise 10: Add Readiness And Liveness Checks

Configure health checks for your app.

For Spring Boot, expose:

```
/actuator/health
```

Example OpenShift probes:

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
```

Exercise 11: Create A Smoke Test Script

Write a small script that verifies the deployed app is basically working.

Example checks:

```
Health endpoint returns 200
Home page/API root responds
Login endpoint is reachable
One database-backed endpoint works
```

Example script:

```
#!/bin/bash

BASE_URL="https://your-app-route"

curl -f "$BASE_URL/actuator/health"
curl -f "$BASE_URL/api/products"
```

Deliverable:

```
scripts/smoke-test.sh
```

Exercise 12: Full Capstone Release Drill

Perform a full release from source code to deployed app.

Checklist:

```
Push code to repository
CI pipeline runs tests
SAST scan completes
Dependency scan completes
Container image builds
Container scan completes
Image is pushed to registry
OpenShift deploys new version
DAST scan runs against deployed route
Smoke tests pass
Release notes are created
```

Deliverable:

```
release-readiness-checklist.md
```

Hands-On Lab

Lab Goal

Take a Java/Spring Boot capstone project and practice a realistic release flow:

```
Build -> Test -> Scan -> Containerize -> Deploy -> Smoke Test
```

Lab Scenario

You are preparing your capstone application for final delivery. Before presenting it, you must prove that:

```
The code builds successfully
Tests pass
Security checks run
The app can be packaged as a container
The app can be deployed
The deployed app is reachable and healthy
```

Prerequisites

```
Java 17 or later
Maven or Gradle
GitHub repository
Spring Boot application
Docker or Podman
Optional: OpenShift CLI, oc
Optional: OWASP ZAP, Trivy, Semgrep
```

Lab 1: Prepare The Application

From your project root, confirm the app builds.

For Maven:

```
mvn clean test
mvn clean package
```

For Gradle:

```
./gradlew clean test
./gradlew build
```

Expected result:

```
Tests pass
A JAR file is created in target/ or build/libs/
```

Create:

```
docs/module51-build-notes.md
```

Add:

```
Build command used
Test result
JAR file name
Any build issues found
```

Lab 2: Add A Health Endpoint

If your app uses Spring Boot Actuator, add this dependency.

Maven:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

In `application.properties` :

```
management.endpoints.web.exposure.include=health,info
management.endpoint.health.probes.enabled=true
```

Run the app:

```
mvn spring-boot:run
```

Test:

```
curl http://localhost:8080/actuator/health
```

Expected result:

```
{"status": "UP"}
```

Lab 3: Run A SAST Scan

Use one static analysis tool.

Example with Semgrep:

```
semgrep scan --config auto .
```

Or with Maven SpotBugs if configured:

```
mvn spotbugs:check
```

Record findings in:

```
docs/module51-sast-report.md
```

Use this format:

```
Tool:
Command:
Number of findings:
High-risk findings:
Medium-risk findings:
False positives:
Fixes applied:
```

Goal:

```
Fix at least one real issue or document why no fix is needed.
```

Lab 4: Create A Dockerfile

Create a `Dockerfile` in the project root:

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build the app first:

```
mvn clean package
```

Build the image:

```
docker build -t capstone-app:module51 .
```

Run it:

```
docker run -p 8080:8080 capstone-app:module51
```

In another terminal:

```
curl http://localhost:8080/actuator/health
```

Expected result:

```
Application starts inside container
Health endpoint returns UP
```

Lab 5: Scan The Container Image

Use Trivy if available:

```
trivy image capstone-app:module51
```

Create:

```
docs/module51-container-scan-report.md
```

Add:

```
Image name:
Scan tool:
Critical vulnerabilities:
High vulnerabilities:
Fixes or mitigations:
Decision: Pass / Fail
```

Suggested rule:

```
Critical vulnerabilities = fail
High vulnerabilities = review before release
Medium/Low = document and track
```

Lab 6: Create A CI Pipeline

Create:

```
.github/workflows/module51-ci.yml
```

Example GitHub Actions workflow:

```
name: Module 51 CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
```

```

- name: Checkout code
  uses: actions/checkout@v4

- name: Set up Java
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: '17'

- name: Run tests
  run: mvn test

- name: Build application
  run: mvn package

- name: Upload JAR
  uses: actions/upload-artifact@v4
  with:
    name: capstone-jar
    path: target/*.jar

```

Push your code and confirm:

```

Workflow starts
Tests run
Build succeeds
Artifact is uploaded

```

Lab 7: Add Security To The Pipeline

Extend the workflow with Semgrep:

```

sast:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Run Semgrep
      uses: semgrep/semgrep-action@v1
      with:
        config: auto

```

Your pipeline should now have:

```

Build/test job
SAST job

```

Optional improvement:

Make deployment depend on both jobs passing.

Lab 8: OpenShift Deployment Files

Create a folder:

openshift/

Create openshift/deployment.yaml :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: capstone-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: capstone-app
  template:
    metadata:
      labels:
        app: capstone-app
    spec:
      containers:
        - name: capstone-app
          image: capstone-app:module51
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
```

Create openshift/service.yaml :

```
apiVersion: v1
kind: Service
metadata:
  name: capstone-app
spec:
  selector:
    app: capstone-app
  ports:
    - protocol: TCP
```

```
port: 8080
targetPort: 8080
```

Create `openshift/route.yaml` :

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: capstone-app
spec:
  to:
    kind: Service
    name: capstone-app
  port:
    targetPort: 8080
```

Apply them if you have OpenShift access:

```
oc apply -f openshift/deployment.yaml
oc apply -f openshift/service.yaml
oc apply -f openshift/route.yaml
```

Check:

```
oc get pods
oc get svc
oc get route
```

Lab 9: Smoke Test The Deployment

Create:

```
scripts/smoke-test.sh
```

Example:

```
#!/bin/bash

BASE_URL="$1"

if [ -z "$BASE_URL" ]; then
  echo "Usage: ./smoke-test.sh <base-url>"
  exit 1
fi

curl -f "$BASE_URL/actuator/health" || exit 1

echo "Smoke test passed"
```

Run:

```
./scripts/smoke-test.sh https://your-openshift-route
```

Expected result:

```
Smoke test passed
```

Final Lab Deliverables

By the end, you should have:

```
docs/module51-build-notes.md
docs/module51-sast-report.md
docs/module51-container-scan-report.md
.github/workflows/module51-ci.yml
Dockerfile
openshift/deployment.yaml
openshift/service.yaml
openshift/route.yaml
scripts/smoke-test.sh
```

Final Readiness Decision

Answer:

```
Is this application ready for final capstone delivery?
```

Use this format:

```
Release Decision: Go / No-Go
```

```
Reason:
```

```
Security Status:
```

```
Build Status:
```

```
Deployment Status:
```

```
Smoke Test Status:
```

```
Known Issues:
```

```
Next Actions:
```

Key Takeaway

Module 51 is about making your capstone deliverable like real software: secure, automated, deployed, and verified.